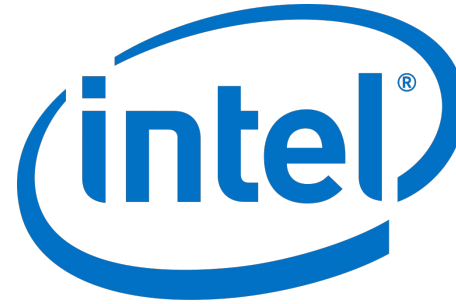

Arquitecturas: X86 y risc-V

Las bases

Conceptos clave:

- Conocer arquitectura risc-V
- Conocer arquitectura x86

Arquitecturas que se verán en la cátedra



x86

risc-v

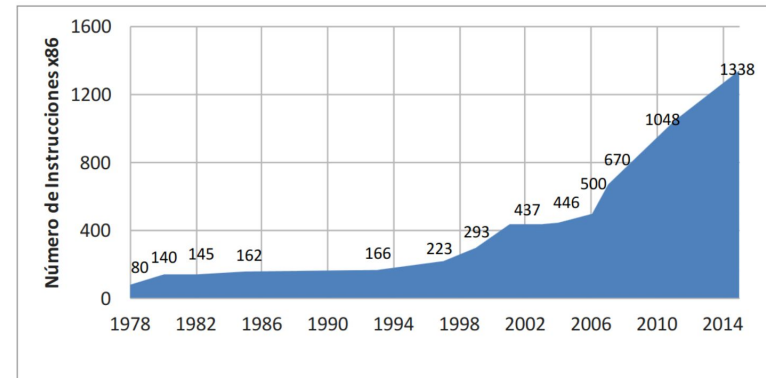
RISC-V es un ISA (*instruction set architecture*) reciente (2010), abierto, minimalista y que arranca de cero informándose de los errores de de ISAs anteriores.

El objetivo de los arquitectos de RISC-V es que sea eficaz para todos los dispositivos de cómputo, desde los más pequeños hasta los más rápidos.

Siguiendo el consejo de von Neumann de hace más de 70 años, este ISA enfatiza **simplicidad** para mantener el costo bajo y muchos registros y velocidad de ejecución transparente para ayudar a compiladores y programadores a resolver problemas importantes de manera eficiente.

ISA	Páginas	Palabras	Horas para leer	Semanas para leer
RISC-V	236	76,702	6	0.2
ARM-32	2736	895,032	79	1.9
x86-32	2198	2,186,259	182	4.5

Manuales de las arquitecturas más usadas

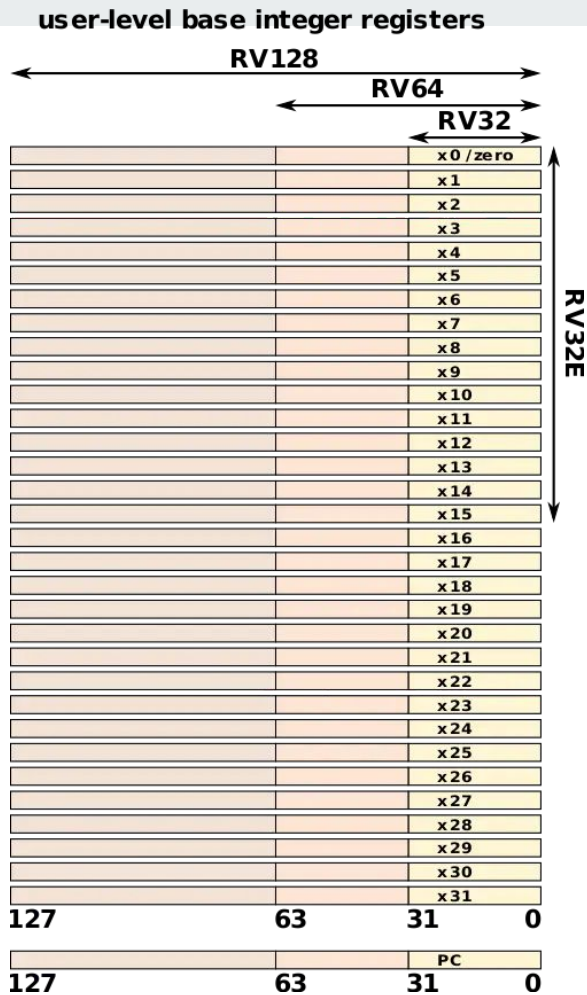


Registros de Propósito General

- **32 Registros de Propósito General (GPRs):**
 - RISC-V define 32 registros de 32 bits (**x0** a **x31**) en su implementación estándar de 32 bits (RV32I). En implementaciones de 64 bits (RV64I) y 128 bits (RV128I), los registros se amplían a 64 bits y 128 bits respectivamente.
 - El registro **x0** es un registro constante que siempre contiene el valor 0.
 - Los registros **x1** a **x31** pueden usarse para operaciones aritméticas, de almacenamiento y para guardar direcciones de memoria.

Registros de Propósito General

Register	ABI Name	Description
x0	zero	Hard-wired zero
x1	ra	Return address
x2	sp	Stack pointer
x3	gp	Global pointer
x4	tp	Thread pointer
x5	t0	Temporary/alternate link register
x6–7	t1–2	Temporaries
x8	s0/fp	Saved register/frame pointer
x9	s1	Saved register
x10–11	a0–1	Function arguments/return values
x12–17	a2–7	Function arguments
x18–27	s2–11	Saved registers
x28–31	t3–6	Temporaries



Modularidad y Extensiones

- **Conjunto de Instrucciones Base (I):**
 - Es el conjunto mínimo de instrucciones necesarias para la mayoría de las aplicaciones, que incluye operaciones aritméticas, lógicas, de control de flujo, y de acceso a memoria.
 - Este conjunto base es obligatorio en todas las implementaciones y define el núcleo funcional de RISC-V.

Extensiones Opcionales:

- **M (Multiplicación y División):** Añade instrucciones para multiplicación y división enteras.
- **A (Operaciones Atómicas):** Proporciona soporte para operaciones atómicas necesarias en programación concurrente.
- **F (Punto Flotante Simple) y D (Punto Flotante Doble):** Añaden soporte para operaciones en punto flotante de 32 y 64 bits respectivamente.
- **C (Compresión de Instrucciones):** Introduce un conjunto de instrucciones de 16 bits para reducir el tamaño del código.
- **V (Vectorial):** Permite operaciones vectoriales para procesamiento de datos en paralelo.
- **B (Bit-Manipulation):** Incluye instrucciones avanzadas para la manipulación de bits.

Direccionamiento de Memoria

- **Espacio de Direcciones:**
 - **RV32:** Usa direcciones de 32 bits, permitiendo un espacio de direcciones de hasta 4 GB.
 - **RV64:** Usa direcciones de 64 bits, permitiendo un espacio de direcciones de hasta 16 exabytes.
 - **RV128:** (Propuesta) Usa direcciones de 128 bits, para un espacio de direcciones extremadamente grande, aunque aún es experimental.

Modos de Dirección:

- **Base+Offset:** Direccionamiento sencillo basado en una base y un desplazamiento.
- **Inmediato:** Se utiliza un valor inmediato (constante) dentro de la instrucción para operar directamente.

Juego de Instrucciones

- **Instrucciones Aritméticas:** **ADD**, **SUB**, **MUL**, **DIV**, **REM** (resto de la división).
- **Instrucciones Lógicas:** **AND**, **OR**, **XOR**, **NOT**, **SLL** (desplazamiento lógico a la izquierda), **SRL** (desplazamiento lógico a la derecha), **SRA** (desplazamiento aritmético a la derecha).
- **Instrucciones de Control de Flujo:** **JAL** (salto y enlace), **JALR** (salto y enlace a registro), **BEQ**, **BNE**, **BLT**, **BGE** (instrucciones de comparación y salto condicional).
- **Instrucciones de Acceso a Memoria:** **LW** (load word), **SW** (store word), **LB**, **SB** (load/store byte), con soporte para diferentes tamaños de datos.

Virtualización de Memoria

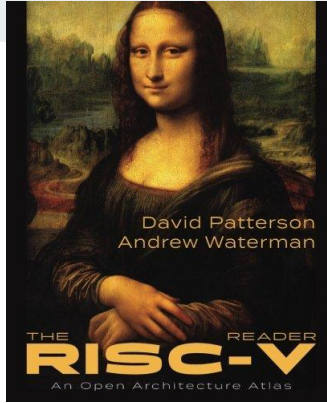
- **Paginación y Segmentación:**
 - RISC-V soporta la paginación de memoria con tamaños de página configurables (4 KB, 2 MB, 1 GB).
 - El ISA define modos de paginación como Sv32 (32 bits), Sv39 y Sv48 (para 64 bits), que gestionan el espacio de direcciones virtual y permiten la traducción a direcciones físicas.
- **Tabla de Páginas:** Utiliza una estructura jerárquica de tablas de páginas para traducir direcciones virtuales a físicas, permitiendo un acceso eficiente y seguro a la memoria.

Interrupciones y Excepciones

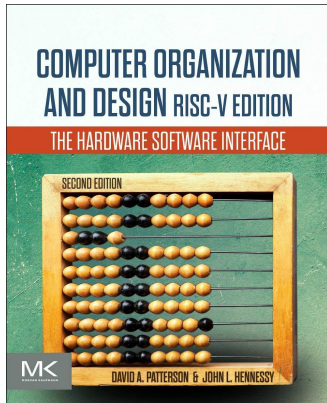
- **Manejo de Excepciones:** RISC-V define un conjunto de vectores de excepciones que manejan situaciones como fallos de página, instrucciones ilegales, y desbordamientos.
- **CSR (Control and Status Registers):** RISC-V usa CSRs para gestionar interrupciones, excepciones y el estado del sistema, permitiendo un control detallado y seguro.

- Soporta tanto implementaciones de 32 bits como de 64 y 128 bits
- 32 registros de proposito general
- 3 modos de ejecución: M, S y U
- Interrupciones
- Virtualización de memoria: paginación
- Enfocado en:
 - Costo
 - Simplicidad
 - Rendimiento

Mas info : [libro](#)



The RISC-V Reader: An Open Architecture Atlas
Paperback – November 7, 2017
by [David Patterson](#), [Andrew Waterman](#)



Computer Organization and Design RISC-V Edition: The
Hardware Software Interface - 2nd Edition
by [David A. Patterson](#), [John L. Hennessy](#)

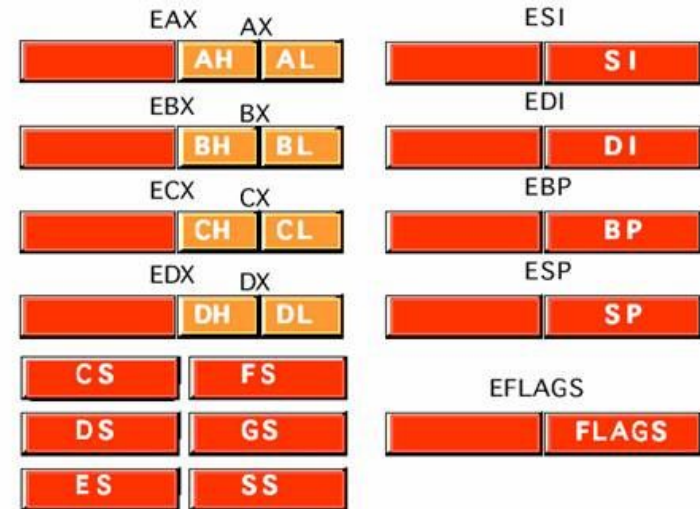
Es una arquitectura de conjunto de instrucciones desarrollada por Intel y utilizada ampliamente en una gran variedad de procesadores, incluyendo los de Intel y AMD. Este ISA es una extensión del original ISA x86 de 16 bits y es fundamental en la historia de los procesadores de propósito general debido a su popularidad y durabilidad en el mercado.

Es de ISA CISC y de micro arquitectura RISC:

“Las instrucciones CISC complejas del ISA x86 son descompuestas internamente en una serie de micro-operaciones más simples (micro-ops) que se asemejan a las instrucciones RISC. Cada micro-op es más sencilla y se puede ejecutar más rápidamente dentro del procesador.”

Registros de Propósito General

- **EAX, EBX, ECX, EDX:** Son los registros de propósito general de 32 bits, que se utilizan para operaciones aritméticas, lógicas, y de almacenamiento temporal. Cada uno de estos registros puede dividirse en partes de 16 bits (AX, BX, CX, DX) y a su vez en partes de 8 bits (AL, AH, BL, BH, etc.).
- **ESI, EDI:** Registros utilizados típicamente para operaciones de apuntado y manipulación de cadenas.
- **EBP (Base Pointer):** Se utiliza comúnmente para apuntar al inicio del marco de la pila en funciones y subrutinas.
- **ESP (Stack Pointer):** Apunta a la parte superior de la pila y es fundamental para operaciones de gestión de la pila como llamadas a funciones y manejo de interrupciones.
- **EIP (Instruction Pointer):** Contiene la dirección de la próxima instrucción a ejecutar.



Segmentos de Memoria

- **CS (Code Segment):** Segmento que contiene las instrucciones del programa.
- **DS (Data Segment):** Segmento que contiene datos del programa.
- **SS (Stack Segment):** Segmento que contiene la pila del programa.
- **ES, FS, GS:** Segmentos adicionales que permiten un direccionamiento más flexible en operaciones con memoria.

Modos de Operación

- **Modo Real:** Compatibilidad con el x86 original de 16 bits, limitando las direcciones a 20 bits y proporcionando acceso directo al hardware.
- **Modo Protegido:** Introducido con el 80386, es el modo nativo de 32 bits que permite acceso a 4 GB de memoria, protección de memoria, multitarea, y control detallado del privilegio del software.
- **Modo Virtual 8086:** Permite la ejecución de aplicaciones de 16 bits dentro de un entorno protegido de 32 bits, combinando la flexibilidad del modo protegido con la compatibilidad hacia atrás.

Juego de Instrucciones

- **Instrucciones Aritméticas y Lógicas:** Operaciones como **ADD, SUB, MUL, DIV, AND, OR, XOR, NOT**, etc.
- **Instrucciones de Control de Flujo:** Instrucciones de salto como **JMP, JE, JNE**, y **LOOP**, así como llamadas a funciones (**CALL**) y retornos (**RET**).
- **Instrucciones de Manipulación de la Pila:** **PUSH, POP, PUSHA, POPA**.
- **Instrucciones de Movimiento de Datos:** **MOV, LEA, XCHG**, etc.
- **Instrucciones de Entrada/Salida:** **IN, OUT** para la comunicación con dispositivos de hardware.
- **Instrucciones SIMD (Single Instruction, Multiple Data):** Conjunto de instrucciones MMX, SSE, y SSE2, que permiten la operación paralela sobre vectores de datos.

Dirección de Memoria

- **Modo de Dirección Lineal:** Las direcciones de 32 bits permiten acceso directo a un espacio de 4 GB de memoria.
- **Segmentación y Paginación:** El modo protegido usa segmentación para definir límites de memoria y paginación para gestionar la memoria virtual, permitiendo la creación de espacios de direcciones virtuales que se traducen a direcciones físicas.

Paginación

- **Páginas de 4 KB:** Paginación estándar donde el espacio de direcciones se divide en bloques de 4 KB.
- **Páginas Grandes:** Opcionalmente, se pueden usar páginas de 4 MB para optimizar la gestión de memoria en ciertos escenarios.

Manejo de Excepciones e Interrupciones

- **Interrupt Descriptor Table (IDT):** Tabla que define manejadores para interrupciones y excepciones, permitiendo un manejo estructurado de eventos del sistema, como fallos de página o interrupciones externas.

INTEL 80386

PROGRAMMER'S REFERENCE MANUAL

1986

Page 1 of 421

Convención de Llamadas

Conceptos clave:

- Convención de Llamadas
- x86
- riscv

Que es la convención de llamadas

Las convenciones de llamada son un conjunto de reglas que le dicen a la computadora cómo pasar información (o "mensajes") entre diferentes partes de un programa cuando necesita que hagan algo

Son las reglas que debe seguir una parte de un programa para comunicarse con otra parte del mismo.

Esos Pasos son:

1. Pasar Argumentos
2. Recibir valor/res de retorno
3. Guardar Información Importante

Call Convention

Hay seis etapas generales al llamar una función [Patterson and Hennessy 2017]:

1. Poner los argumentos en algún lugar donde la función pueda acceder a ellos.
2. Saltar a la función .
3. Reservar el espacio de memoria requerido por la función, almacenando los registros que se requiera.
4. Realizar la tarea requerida de la función.
5. Poner el resultado de la función en un lugar accesible por el programa que invocó a la función, restaurando los registros y liberando la memoria.
6. Dado que una función puede ser llamada desde varias partes de un programa, retornar el control al punto de origen .

Componentes de la Convención de Llamada en RISC-V

Registros para pasar argumentos (argument registers):

- RISC-V usa los registros **a0** a **a7** para pasar hasta ocho argumentos enteros o de punto flotante a una función.
- Para argumentos de punto flotante, si hay una unidad de punto flotante disponible (hardware de punto flotante), se utilizan los registros **fa0** a **fa7**.
- Si hay *más de ocho argumentos*, los adicionales se pasan en la pila, un área de memoria que se utiliza para almacenar datos temporales.

Registros para devolver resultados (return registers):

- Los resultados de las funciones se devuelven usando los registros **a0** y **a1** para valores enteros o de punto flotante. Si se devuelve un número de punto flotante y hay soporte de hardware para punto flotante, se usan los registros **fa0** y **fa1**.

Uso de la pila (stack usage):

- La pila es una estructura de datos en la memoria que crece hacia abajo (hacia direcciones de memoria más bajas) a medida que se añaden datos.
- En RISC-V, el registro **sp** (stack pointer) apunta al inicio de la pila. Cuando se necesitan más de ocho argumentos o cuando los registros deben preservarse, esos datos se guardan en la pila.

Componentes de la Convención de Llamada en RISC-V

Registros preservados y temporales:

- Registros preservados (s0 a s11): Estos registros **deben conservar su valor original durante la ejecución de una función**. Esto significa *que si una función los usa, debe guardar su valor original (por ejemplo, en la pila) y restaurarlos antes de retornar*.
- Registros temporales (t0 a t6): *Estos registros pueden ser usados libremente por cualquier función*. No es necesario guardar su valor anterior, ya que se consideran "volátiles" y pueden cambiar con cada llamada a función.

Alineación de la pila:

- La pila en RISC-V debe estar alineada a 16 bytes. Esto significa que, al realizar llamadas a funciones o manipular la pila, el puntero de pila (sp) debe siempre apuntar a una dirección que sea múltiplo de 16. Esto ayuda a optimizar el acceso a memoria y a evitar problemas de rendimiento o errores.

Componentes de la Convención de Llamada en RISC-V

Ejemplo 1: hasta 8 argumentos

`sum(int a, int b)`

`sum:`

```
add a0, a0, a1 # Suma el contenido de a0 y a1 (segundo argumento), almacena el resultado en a0
ret           # Devuelve de la función
```

Ejemplo 2: Mas de 8 argumentos

`foo(int a1, int a2, ..., int a10)`

```
addi sp, sp, -16    # Reserva espacio en la pila (suponiendo que necesitábamos 16 bytes más para nuestros dos argumentos adicionales)
sw a9, 0(sp)        # Guarda el noveno argumento en la pila
sw a10, 4(sp)       # Guarda el décimo argumento en la pila
call foo            # Llama a la función foo
addi sp, sp, 16     # Restaura el puntero de pila después de la llamada
```

Componentes de la Convención de Llamada en RISC-V

Ejemplo 3: Uso de registros reservados dentro de una llamada

Función principal (main) que llama a 'compute'

```
main:                # Supongamos que s0 tiene un valor importante aquí
    li s0, 100        # Cargamos el valor 100 en el registro s0
    call compute      # Llamamos a la función compute
    # Aca se espera que s0 siga teniendo el valor 100
    ...
    ret               # Retorno de la función main
```

Función 'compute' que utiliza el registro preservado s0

```
compute:
    addi sp, sp, -16   # Reserva espacio en la pila para guardar s0 (16 bytes para alineación de 16 bytes)
    sw s0, 0(sp)       # Guarda el valor original de s0 en la pila

    # Realiza algunos cálculos que usan s0
    li s0, 200         # Cambia el valor de s0
    # Aquí s0 se utiliza para algunos cálculos, por ejemplo:
    add s0, s0, a0      # Suma el valor en s0 con el argumento en a0

    # Restaura el valor original de s0 desde la pila
    lw s0, 0(sp)       # Carga el valor original de s0 desde la pila
    addi sp, sp, 16     # Restaura el puntero de pila (sp) a su posición original
    ret                # Retorna de la función compute
```

Call Convention: risc-v

La clave es tener algunos registros que no se garantiza que conserven su valor a través de una llamada a una función, llamados temporary registers, y otros que sí se conservan, llamados saved registers.

Saved registers risc-v:

Sp

S0-s11

Fs0-1

Fs2-11

Register	ABI Name	Description	Saver
x0	zero	Hard-wired zero	—
x1	ra	Return address	Caller
x2	sp	Stack pointer	Callee
x3	gp	Global pointer	—
x4	tp	Thread pointer	—
x5-7	t0-2	Temporaries	Caller
x8	s0/fp	Saved register/frame pointer	Callee
x9	s1	Saved register	Callee
x10-11	a0-1	Function arguments/return values	Caller
x12-17	a2-7	Function arguments	Caller
x18-27	s2-11	Saved registers	Callee
x28-31	t3-6	Temporaries	Caller
f0-7	ft0-7	FP temporaries	Caller
f8-9	fs0-1	FP saved registers	Callee
f10-11	fa0-1	FP arguments/return values	Caller
f12-17	fa2-7	FP arguments	Caller
f18-27	fs2-11	FP saved registers	Callee
f28-31	ft8-11	FP temporaries	Caller

Componentes de la Convención de Llamada en x86

Registros preservados y temporales:

- Registros preservados (s0 a s11): Estos registros **deben conservar su valor original durante la ejecución de una función**. Esto significa *que si una función los usa, debe guardar su valor original (por ejemplo, en la pila) y restaurarlos antes de retornar*.
- Registros temporales (t0 a t6): *Estos registros pueden ser usados libremente por cualquier función*. No es necesario guardar su valor anterior, ya que se consideran "volátiles" y pueden cambiar con cada llamada a función.

Alineación de la pila:

- La pila en RISC-V debe estar alineada a 16 bytes. Esto significa que, al realizar llamadas a funciones o manipular la pila, el puntero de pila (sp) debe siempre apuntar a una dirección que sea múltiplo de 16. Esto ayuda a optimizar el acceso a memoria y a evitar problemas de rendimiento o errores.